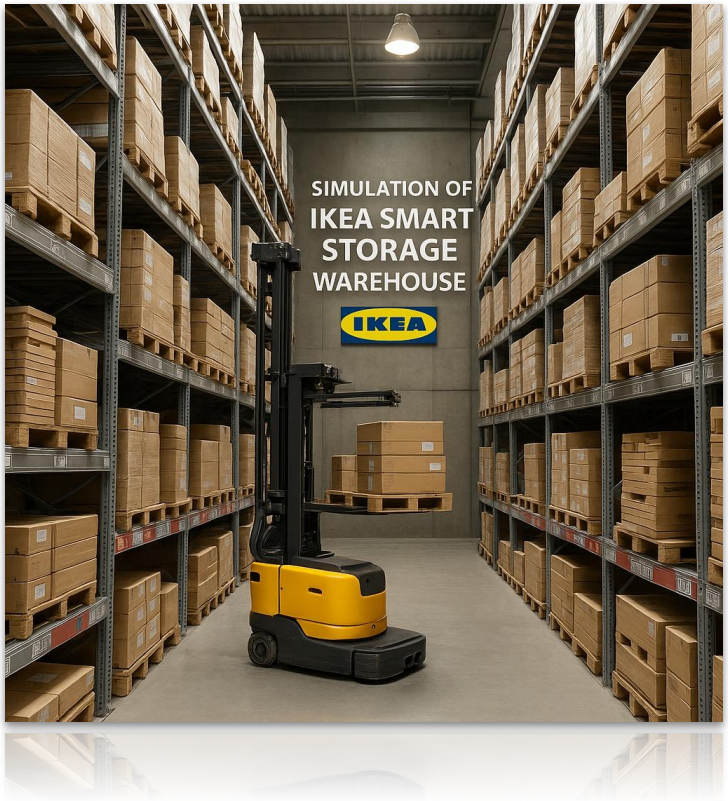
Group 13

Automated IKEA Warehouse

Selected Prototype

Prototype:

- Automated IKEA Warehouse ©
- Furniture as items
- Robots for operations
- Charging stations for robots
- Administrators who add, move and remove items



Roles in Team

Alisa – Charging system developer (Robot management, Charging management)

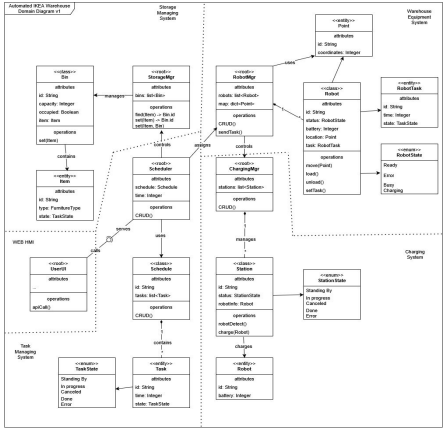
Akanksha – Frontend & storage system developer (domain model, UI, Storage management)

Mikhail – System architect & backend integrator (Scheduler management, architecture)

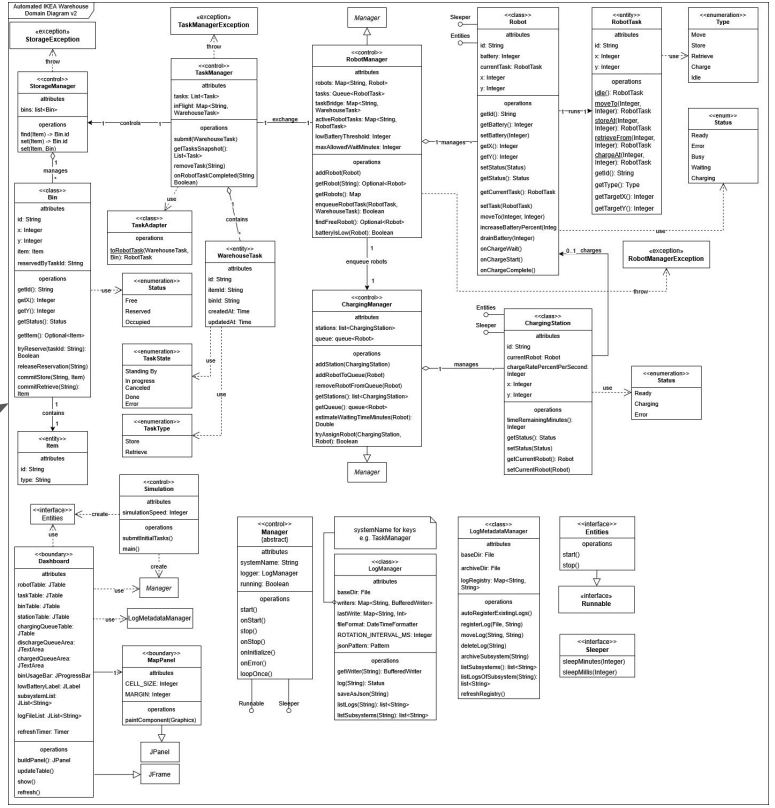
Domain Class Diagram

Lines of dots separate subdomains. Each subdomain contains a root class, which is represented uniquely.

The system assumes asynchronous simulation of all subdomains.



getsBigger



Storage Management

Purpose
Manages all warehouse bins and stored items, ensuring item placement and retrieval occur safely and consistently.

Responsibilities

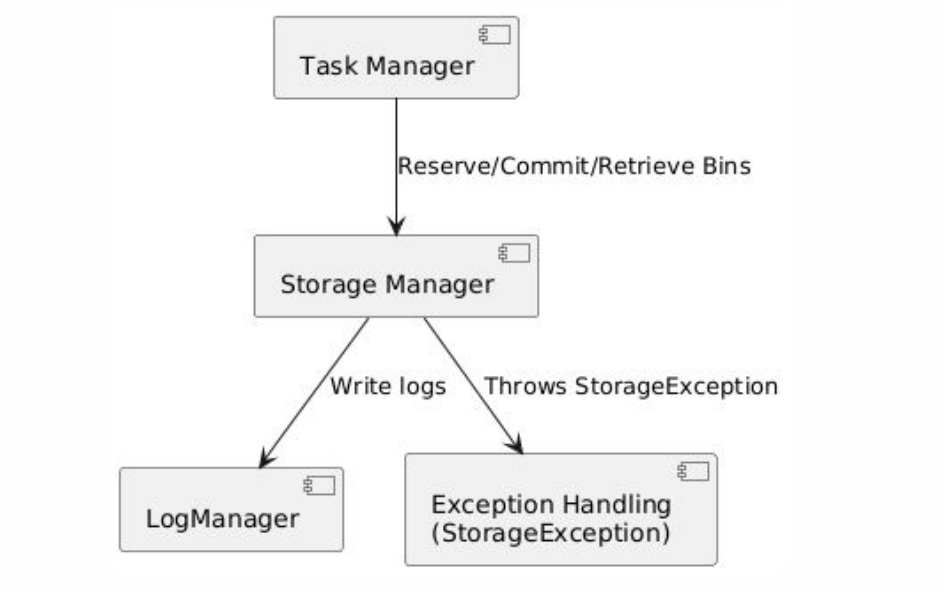
- Maintain global bin state (FREE, RESERVED, OCCUPIED)
- Reserve bins for incoming STORE tasks
- Validate and locate bins for RETRIEVE tasks
- Apply final storage changes after robot actions (commit store / commit retrieve)
- Safeguard warehouse data integrity

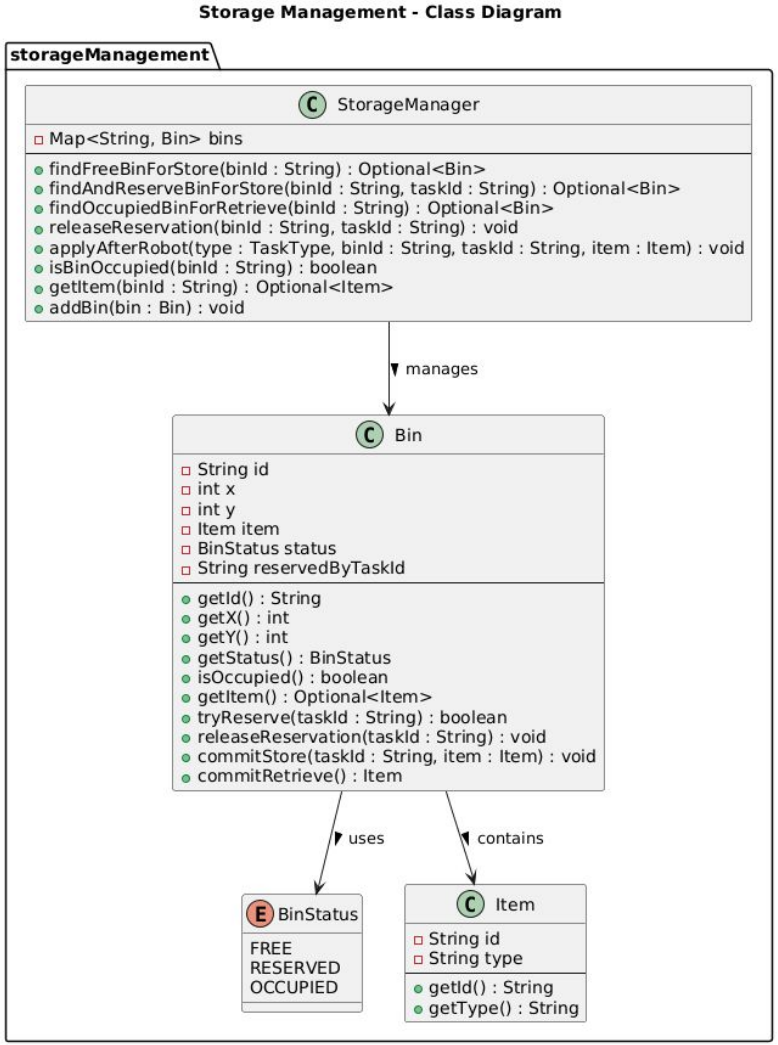
Key Classes

- **StorageManager** - central controller for bin lookup, reservation, and state updates
- **Bin** - represents storage locations with coordinates, status, and item data
- **Item** - stored object representation (ID + type)

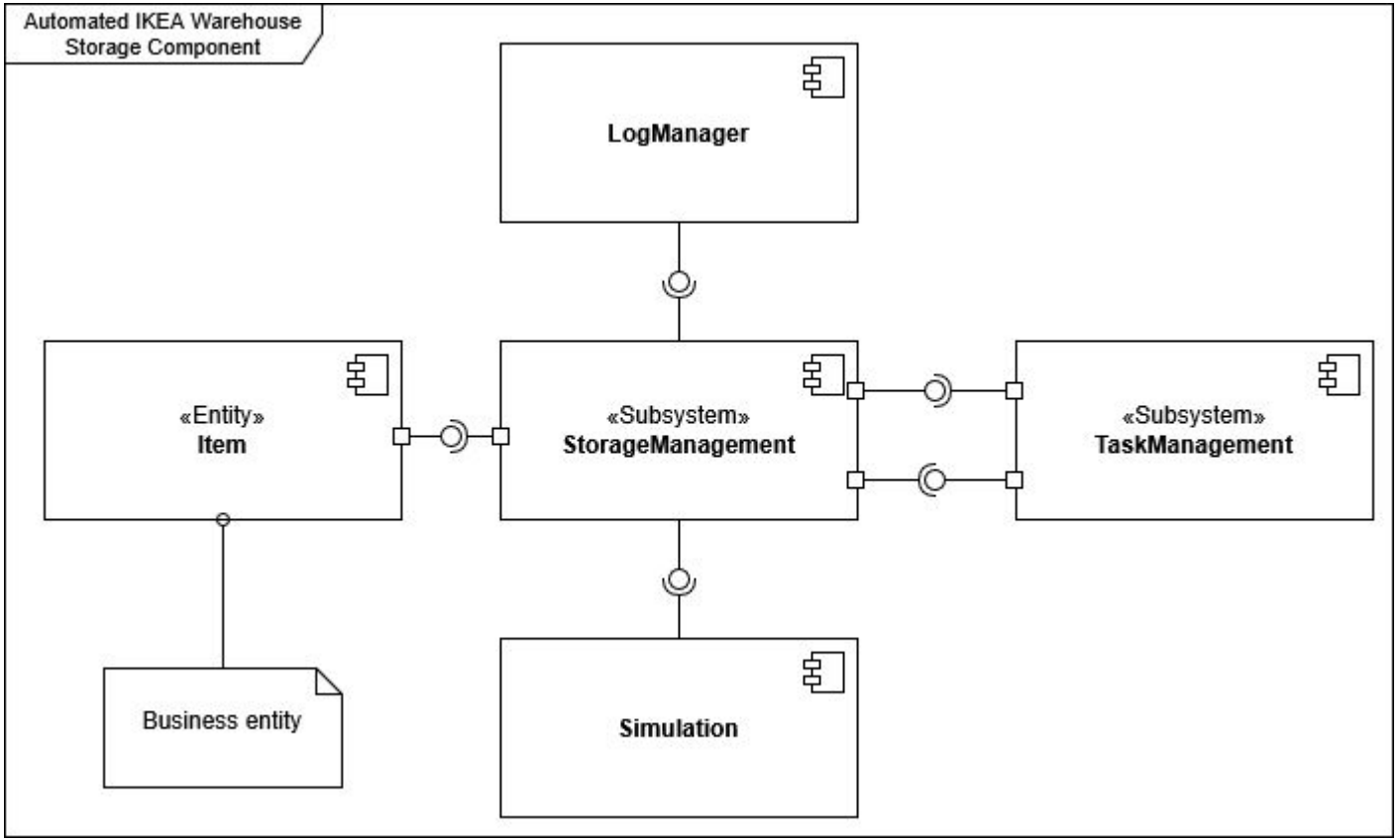
Storage Management

Component Diagram





Storage Management



Storage Management

Threading & Concurrency

- **StorageManager** runs in its own Manager thread, though operations are lightweight
- Bin operations are synchronized to prevent race conditions
- Collaborates with RobotManager & TaskManager through thread-safe workflows
- Ensured state consistency during concurrent STORE and RETRIEVE task processing

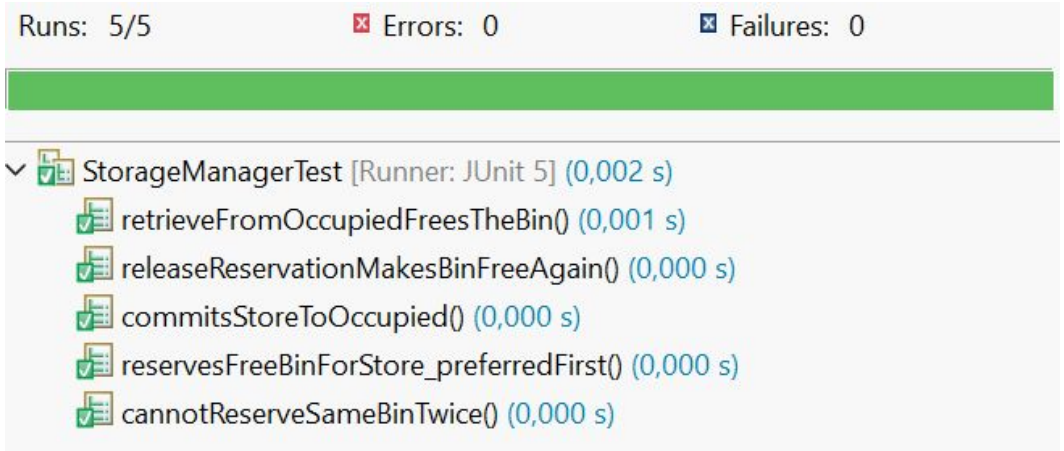
Storage Management

Tests

Storage Management tests verify bin reservation, state transitions, and safe interaction with TaskManager and RobotManager.

Unit Tests (RobotManagerTest)

retrieveFromOccupiedFreesTheBin	ensures RETRIEVE operations free the bin
releaseReservationMakesBinFreeAgain	checks reservation rollback
commitsStoreToOccupied	validates correct STORE commit behavior
reservesFreeBinForStore_preferredFirst	checks preferred-bin reservation logic
cannotReserveSameBinTwice	ensures no double-reservation



Robot Management

Purpose
Coordinates and controls all warehouse robots, ensuring tasks are executed safely and efficiently.

Responsibilities

- Assigns and schedules robot tasks (movement, storage, retrieval, charging)
- Monitors robot state and battery levels
- Communicates task outcomes back to Task Manager
- Ensures robots operate safely and without conflicts

Key Classes

- **RobotManager** - core scheduler, assigns robot tasks, runs management loop
- **Robot** - autonomous uni that executes movement, storage, retrieval, charging
- **RobotTask** - immutable task description (move, store, retrieve, charge, idle)

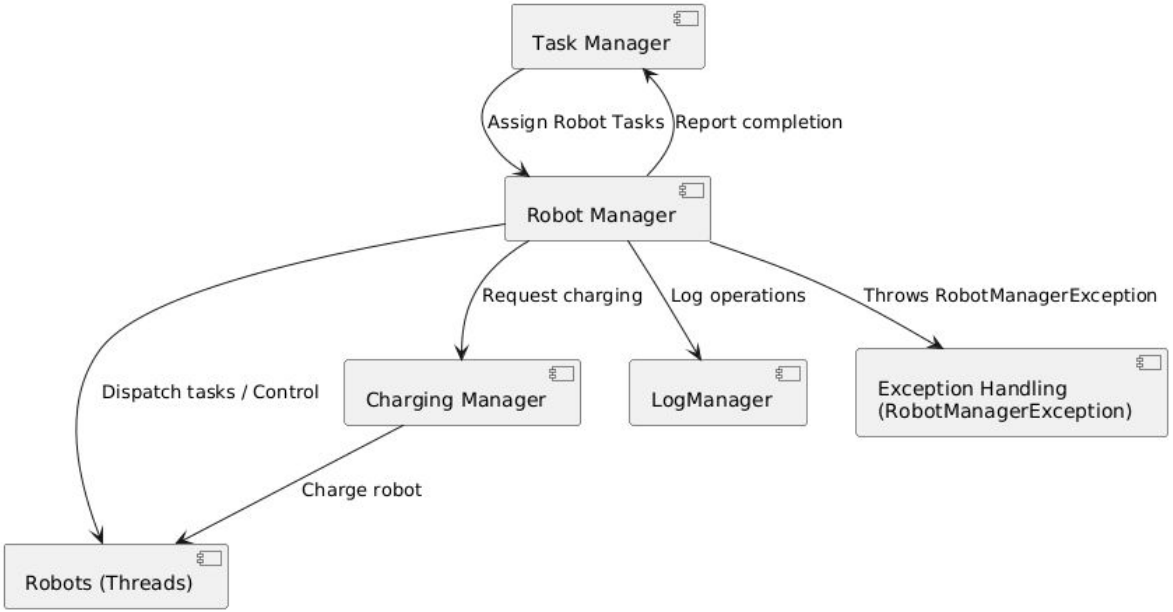
Robot Management

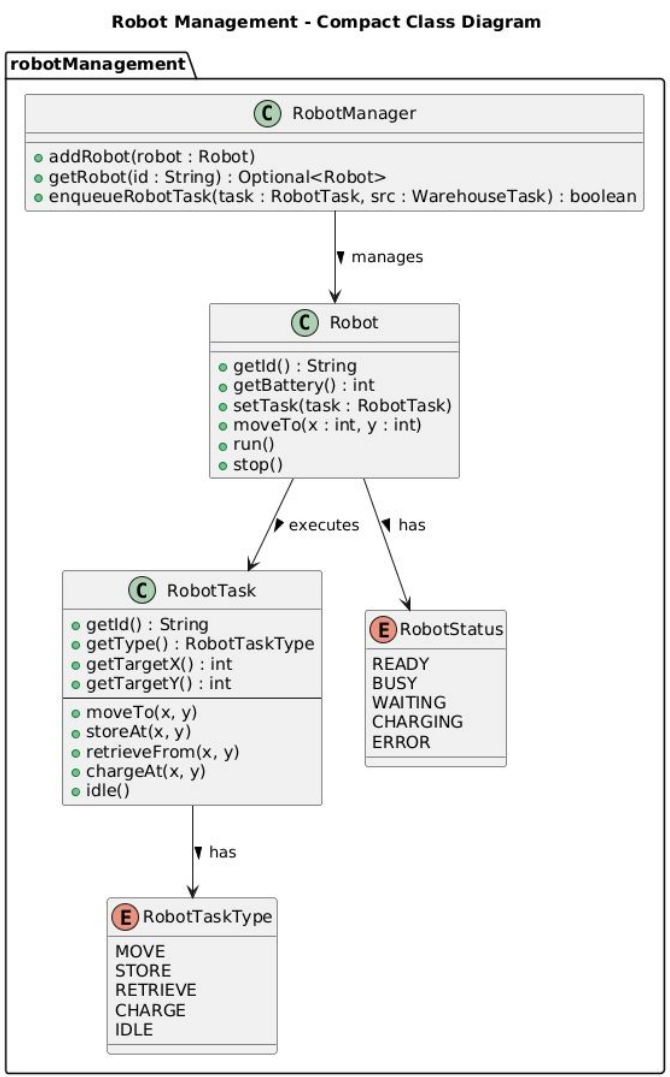
Threading & Concurrency

- **Each Robot runs in its own thread**, executing its assigned task asynchronously
- **RobotManager runs a separate loop thread,** dispatching tasks & monitoring state
- Uses **internal synchronized sections** or atomic checks to avoid conflicting assignments
- Thread interaction occurs when robots **callback to** RobotManager upon task completion

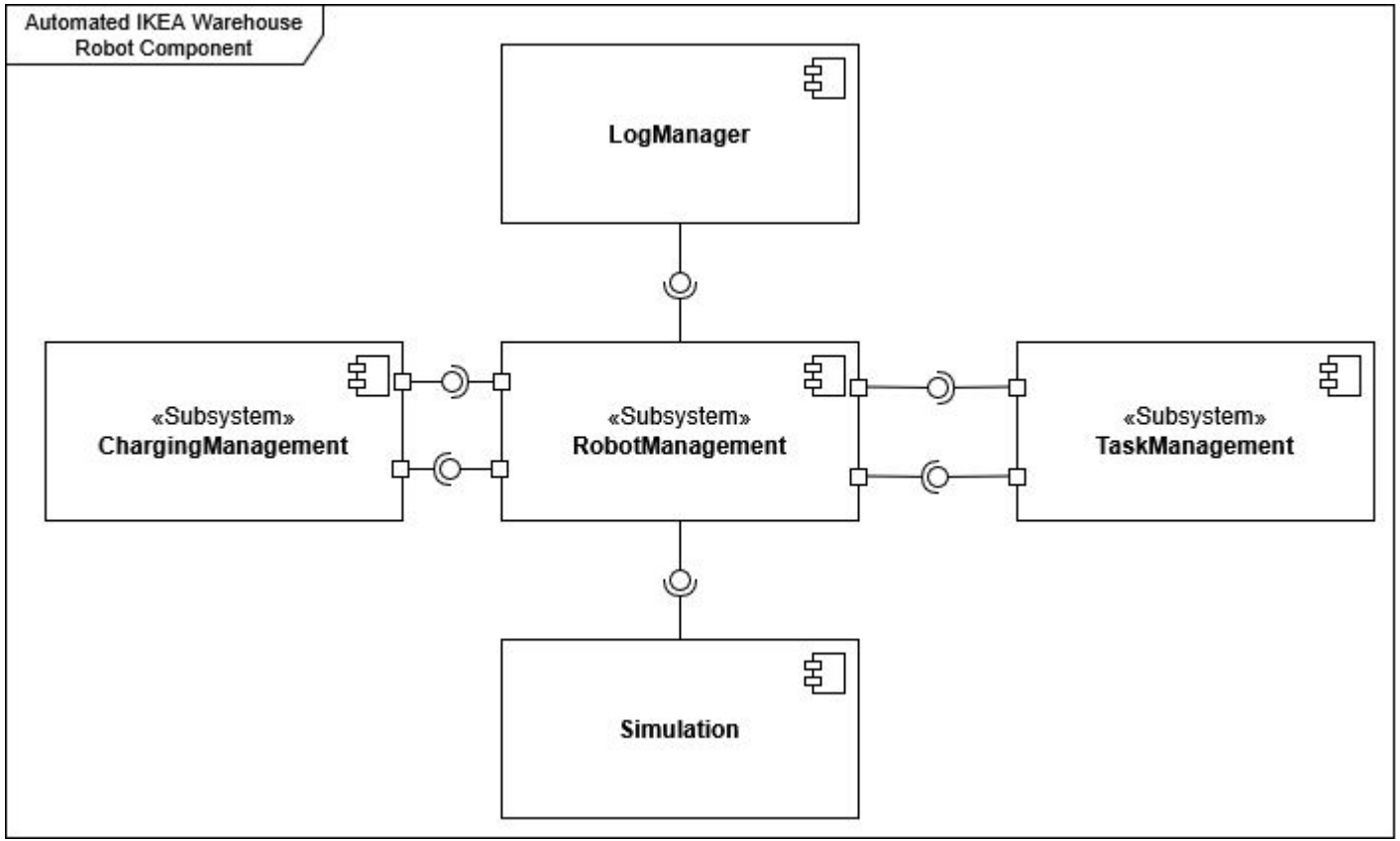
Robots Management

Component Diagram





Robots Management

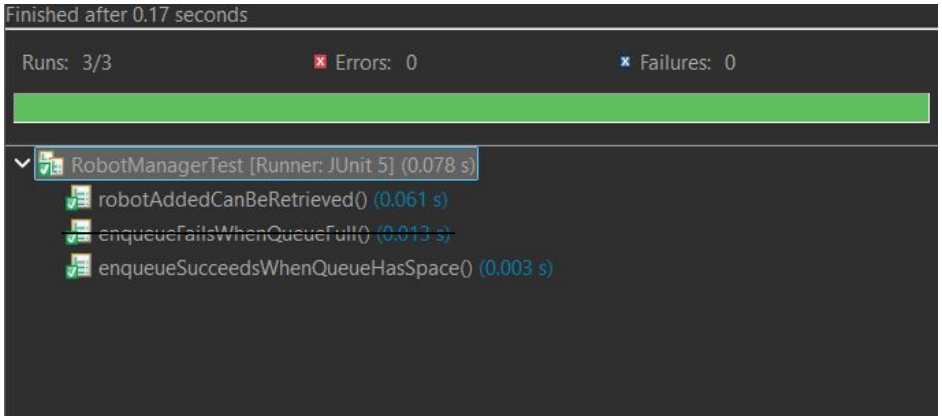


Robot Management

Tests

Robot Management is validated through **unit tests** & **integration test** ensuring correct assignment, queue behaviour, & charging coordination

Unit Tests (RobotManagerTest)



robotAddedCanBeRetrieved	Ensures a robot added to the manager can be retrieved
enqueueFailsWhenQueueFull	Verifies that exceeding queue capacity throws an exception
enqueueSucceedsWhenQueueHasSpace	Confirms a robot task is accepted when queue has space

Charging Management

Purpose
Coordinates and controls all warehouse charging infrastructure, ensuring robots are charged.

Responsibilities

- Maintains and manages the set of available charging stations.
- Maintains a queue of robots waiting for charging.
- Estimates waiting time per station and decides whether a robot should be sent to charge.
- Assigns robots in the queue to specific ChargingStations when slots become free.
- Updates charging-related RobotTasks with the coordinates of the assigned ChargingStation.
- Coordinates with RobotManager / TaskManager to avoid unnecessary charging tasks and reduce idle time.

Key Classes

- **ChargingManager** – central coordinator for all charging operations; owns the charging queue, list of stations, and the logic that decides if/when a robot should go to charge and to which station.
- **ChargingStation** – represents a physical charging point in the warehouse; tracks position, occupancy, and whether it is available for a new robot to dock and charge.

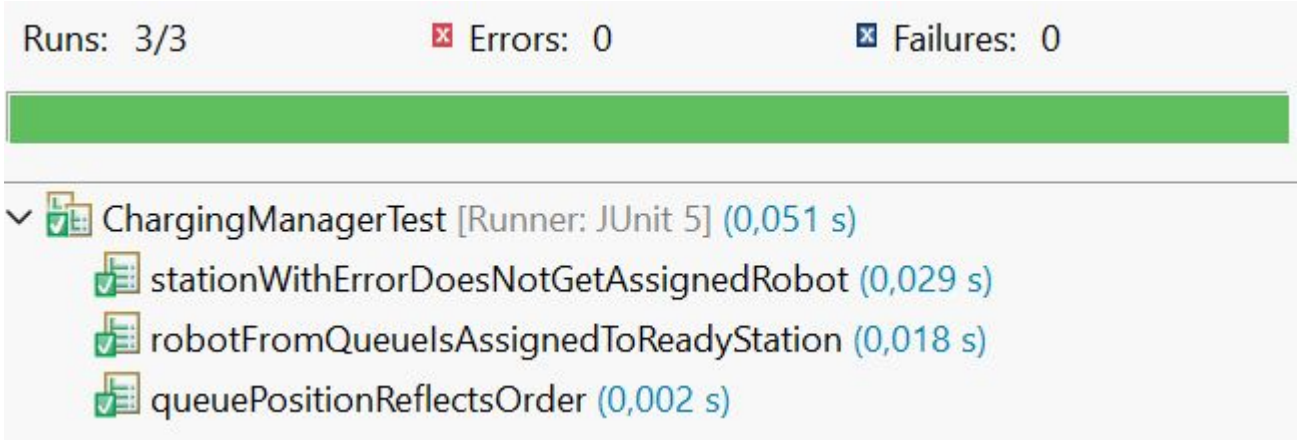
Charging Management

Threading & Concurrency

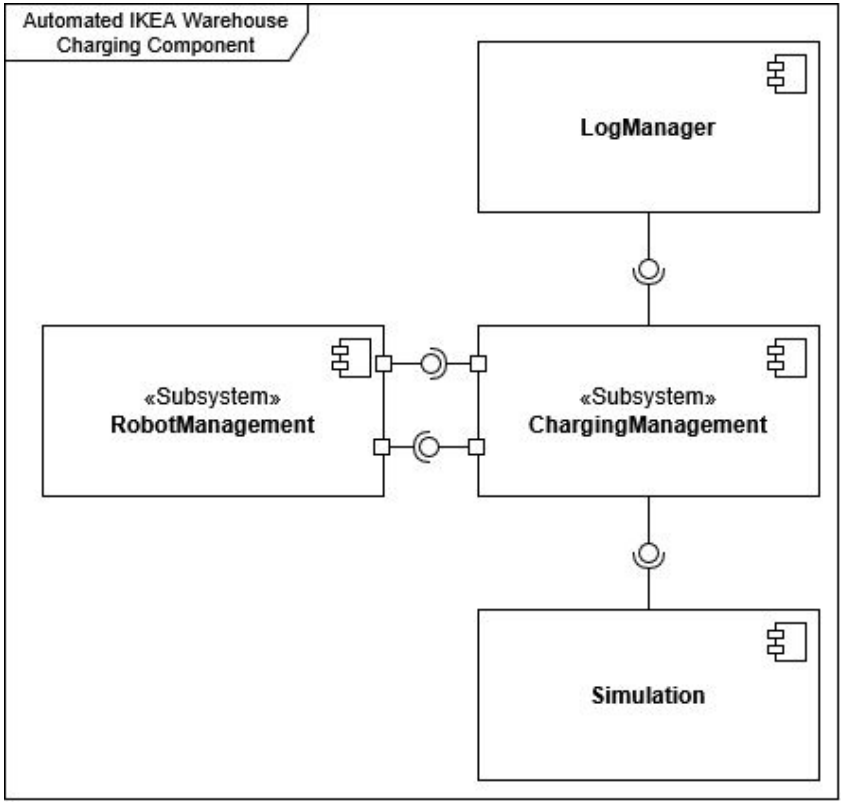
- **Each ChargingStation runs in its own thread**, charging its assigned robot asynchronously.
- **ChargingManager runs a separate loop thread,** assigning robots to stations.
- ChargingManager **provides synchronized access to its internal queue** and **station list** to avoid race conditions when robots are added/removed or assigned.

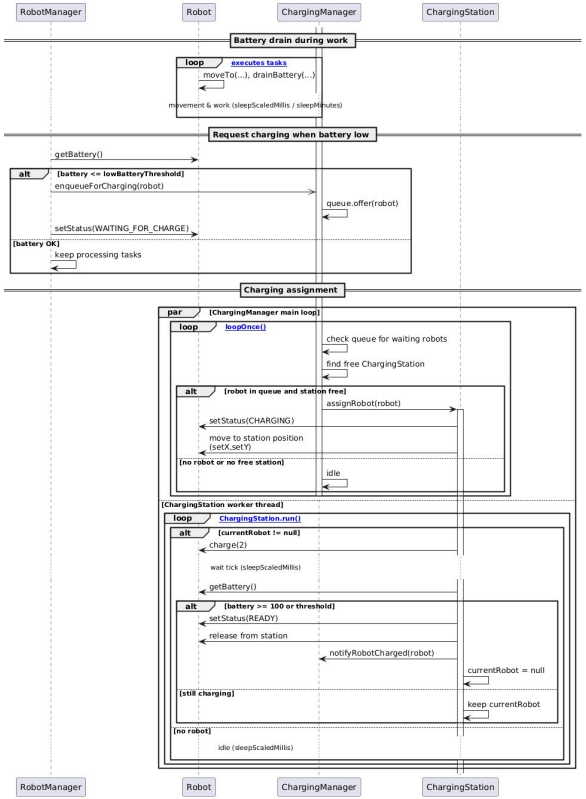
Charging Management

ChargingManagerTest



Charging Management UML





Task Management

Purpose
Coordinates and controls all warehouse tasks, turning high-level requests into executable work for robots, storage, and charging systems.

Responsibilities

- Accepts and validates incoming requests (e.g. store, retrieve, move, charge) via TaskAdapter
- Creates and enqueues WarehouseTask instances with the correct TaskType and initial TaskState
- Maintains the central task queue and decides which task should be processed next
- Tracks full task lifecycle (created → queued → in progress → completed / failed)
- Delegates execution to RobotManager (and indirectly to StorageManager / ChargingManager)
- Handles failures from collaborators (e.g. StorageManager) and wraps them in TaskManagerException with proper chained causes

Key Classes

- **TaskManager** – central coordinator of all warehouse tasks; owns the task queue, lifecycle transitions, and delegation to other managers
- **WarehouseTask** – immutable or semi-immutable description of a single logical task with type, target bin/coordinates, and current state
- **TaskType** – enum describing what kind of work the task represents (STORE, RETRIEVE, MOVE, CHARGE, etc.)
  TaskState – enum representing the lifecycle state of a task (CREATED, QUEUED, IN_PROGRESS, COMPLETED, FAILED, etc.)
- **TaskAdapter** – adapter/facade that converts external or UI/API-level requests into internal WarehouseTask objects consumed by TaskManager

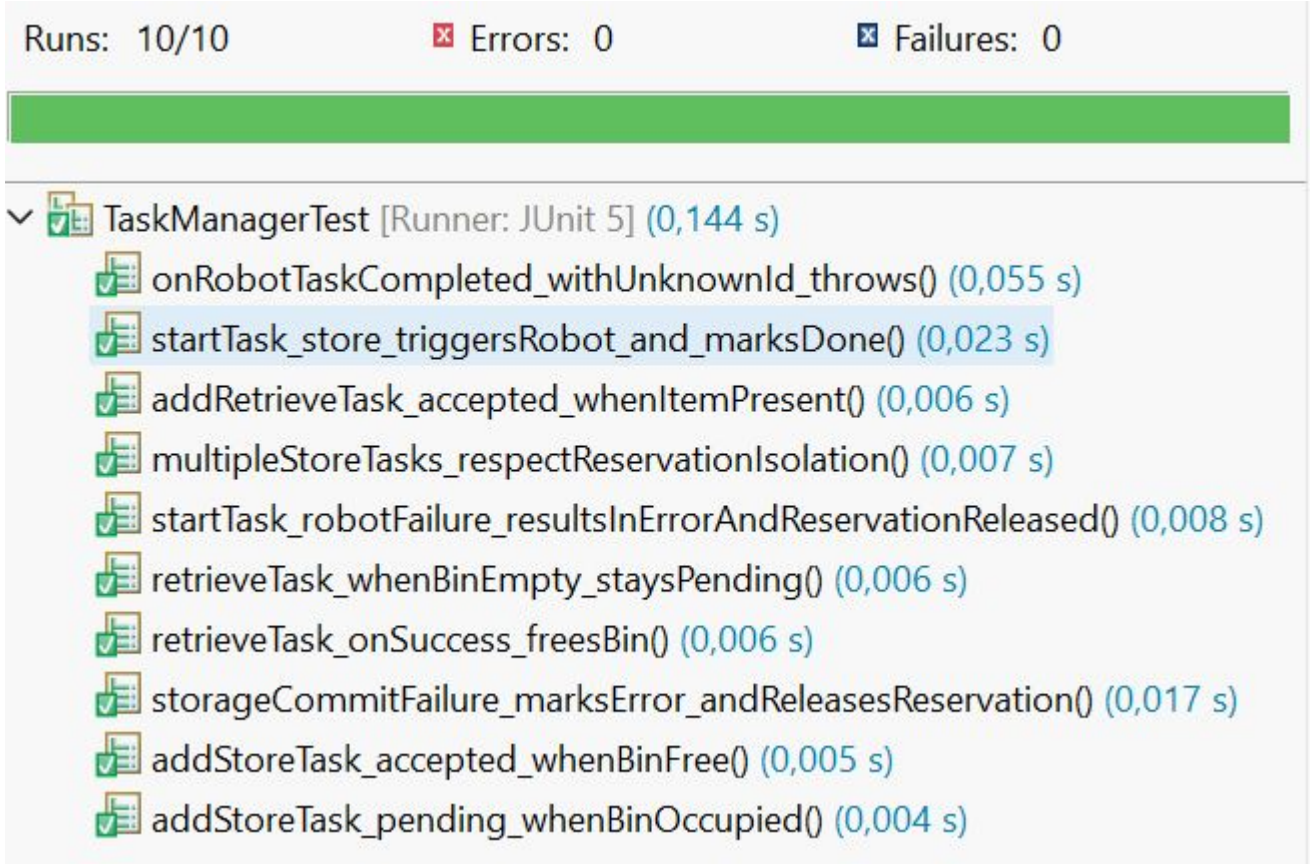
Task Management

Threading & Concurrency

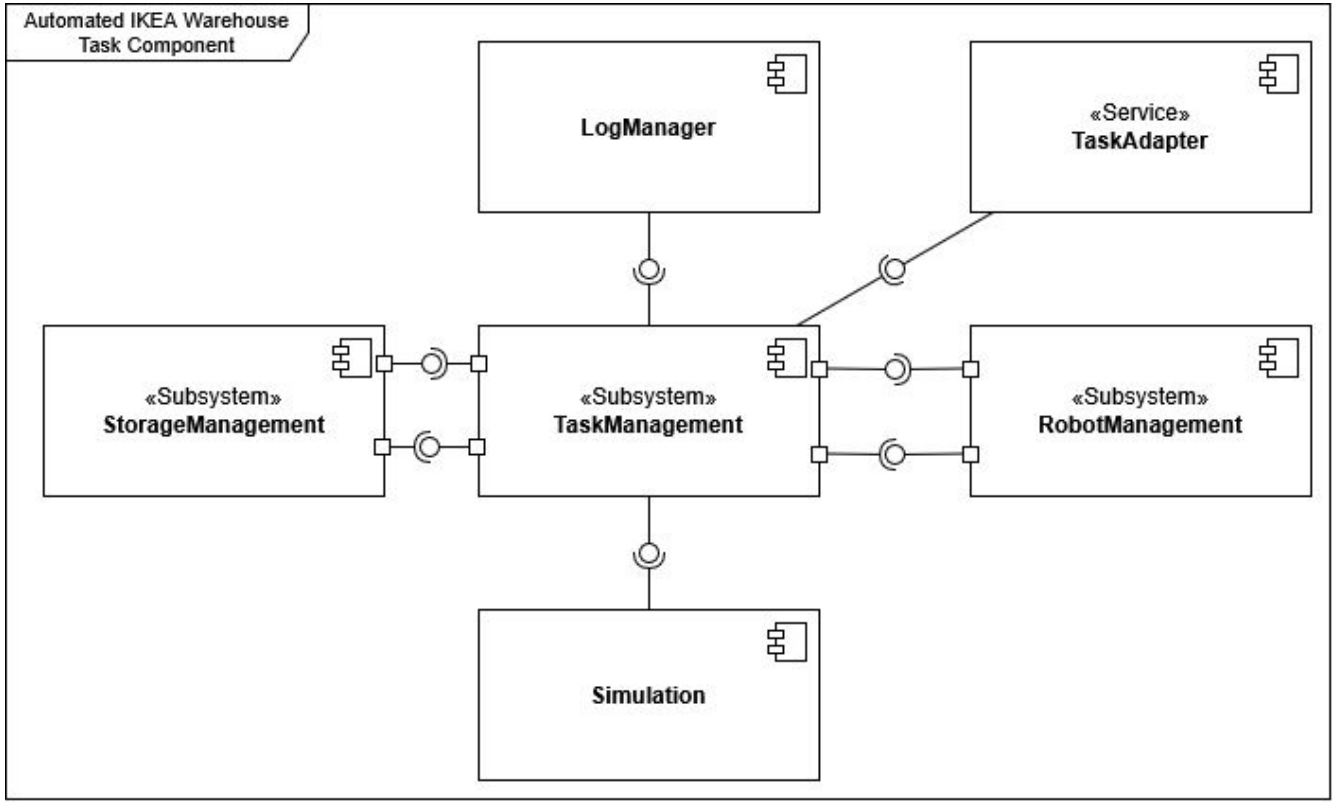
- **TaskManager runs a separate loop thread,** accepting new tasks, selecting the next suitable task and handing it off for execution.
- **TaskManager provides synchronized access** to the **internal task queue** and state transitions to avoid race conditions.
- **Thread interaction** occurs when robots report task completion or failure, allowing TaskManager to update TaskState and possibly enqueue follow-up tasks (e.g. charging).

Task Management

TaskManagerTest



Task Management UML



Dashboard

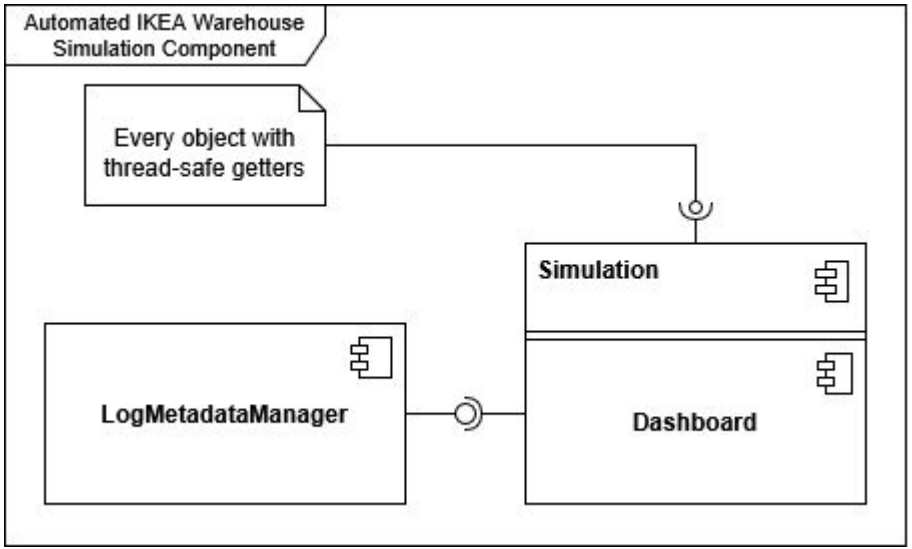
Dashboard – a Swing-based monitoring UI that visualizes the entire warehouse in real time.

Displays robots, storage bins, tasks, charging stations, and key statistics in separate tabs, including a live map view of the warehouse layout.

The Dashboard reads state from RobotManager, StorageManager, TaskManager, and ChargingManager, and integrates with the logging system so subsystem logs could be inspected during the simulation.

Additional components

- **exceptionHandler;**
- **logging** package with **LogManager;**
- abstract **Manager** class;
- main **Simulation** class.



Project integration tests

- **taskMngStoreTaskEndToEndTest**– only the storage-related slice: bin occupancy, item inserted;
- **taskMngRetriveTaskEndToEndTest** – only the storage portion: bin emptied, item removed;
- **chargingMngEndToEndTest** – Validates full charging cycle with robot thread;
- **robotMngAndChargingMngEndToEndTest** – ensures RobotManager and ChargingManager coordinate charging correctly.

The End of 👍

Automated IKEA Warehouse